ME 461 - Let Pico move it - part II

Recall:

Pinout of Pi Pico is also provided as a reference in Figure 1.

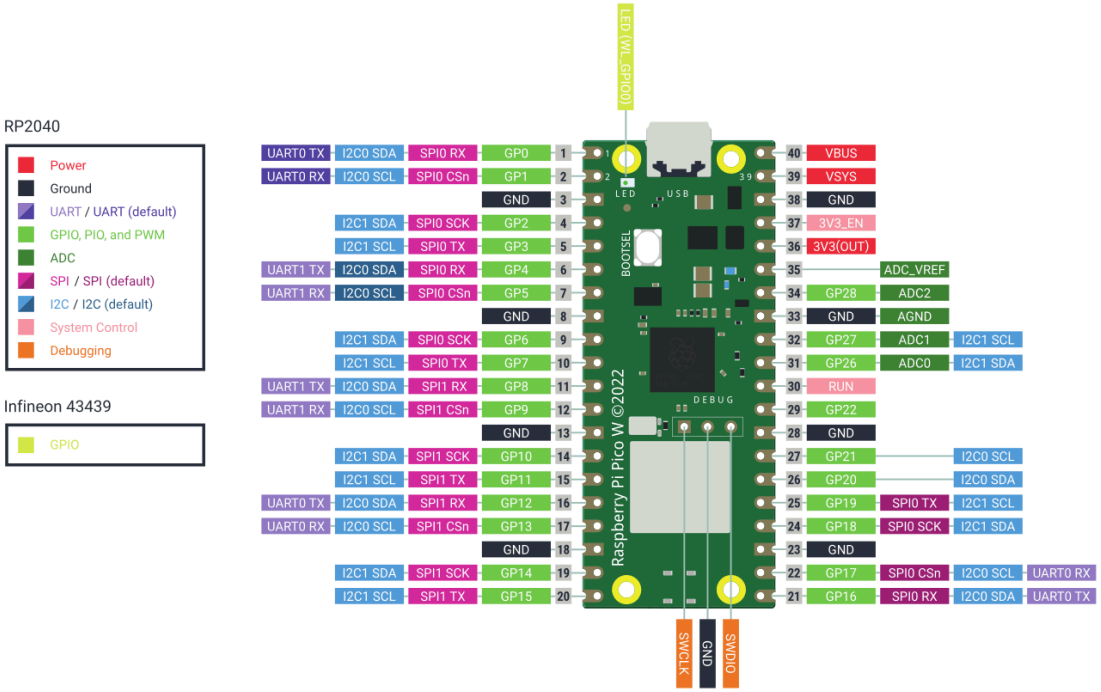

Figure 1: Raspberry Pi Pico W pinout

Rules of thumb: - would not hurt to remember 🙂

- Your code should
  - not crash no matter what
  - include sufficient comments
  - be as lean and short as possible
  - not have self-repeating parts
  - run smoothly and be responsive
- Try to use cut to length wires. Slayed network cables are a good single strand source.
- Everytime you take your breadboard etc out from your backpack or its container, manually check connections to make sure nothing is loose or disconnected. You will get faster in time :)
- Do not modify your circuit while it is powered on
- Do not rush into anything
- If something does not work as expected:
  - it is most probably not a bug but you are missing something.
  - Re-check the cabling and make sure that there are no loose connections
- If you feel a different odor coming from around your breadboard, power it down right away, and do not directly touch things to find out what happened.
- Based on definitions in Labs / Assignments, get used to taking incentives and making reasonable assumptions when certain details are not provided.

What 2 do?

STEP 1: Get ready

Signals in the form of square waves are commonly used in mechatronic systems for several purposes. One application of such periodic signals is in motor actuation. In this lab we will focus on RC Servo motors (will be referred to as *RC Servo* in the remainder of this document). An RC Servo is not simply a DC motor. Indeed, it is a combination of a DC motor, a gear train, a position sensor (commonly a potentiometer is used) and control circuitry as illustrated in Figure 2.

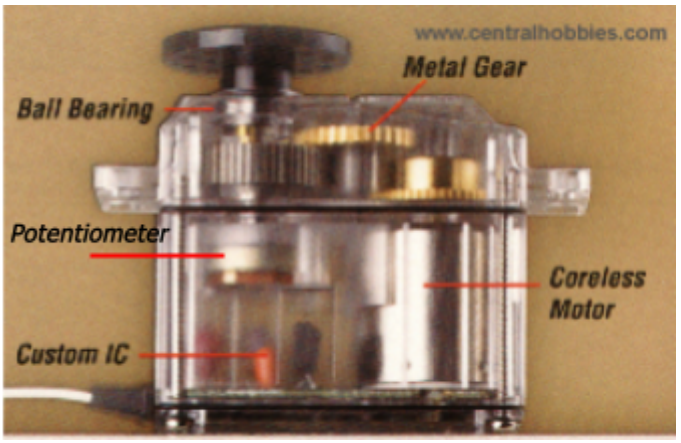

Figure 2. Components of a RC Servo [www.centralhobbies.com]

RC servos are small actuators widely used in remotely controlled (RC) toys (i.e. cars, planes, helicopters and boats). These are compact actuation mechanisms that provide an easy way of achieving position control in small scale applications where precision and accuracy is not critical. Each RC servo is composed of a small DC motor attached to a gear train, a potentiometer and a simple controller. The gear reduction provides a reasonable amount of torque compared to their size. As the name implies, the RC servo has a sensor that tells the current position of the shaft to the controller that is built into this system, namely the potentiometer. The built-in controller compares the current position (measured from the pot) with the target (sent from uController such as Arduino, Pi Pico, ESP32, or even SBCs such as Raspberry Pi 3-4 etc) and creates a control signal that will move the output shaft to a position that decreases this difference. By now if you do not already know, you should have started wondering how we can set the reference position (in other words, how to tell the motor where to go). The reference position is provided by using a PWM (pulse width modulated) signal. Almost all of the RC servos that you will find on the market will have a range of 0-180^{o}. The built-in controller of RC servos control the rotor position within this range. Generally, one end of this range is referred to as –90^{o} and the other as +90^{o} (even though there is no harm if you prefer 0-180^{o}). Figure 3 illustrates the ideal working conditions of RC servos. As shown, RC servos can be rotated to specific positions by sending PWM signals, the ON (high) time of which determines the target position of the motor. In order to reach and maintain a position this PWM signal has to be **continuously sent** to the motor via its signal pin. Ideally the high time of these signals will change between 1 to 2 ms. Low time of these signals can be anywhere between 10 to 25 ms. Use of 10 ms for low time is recommended. Even though ideal high times should be within [1-2]ms range, in practice this range will slightly change and in this lab you will determine these limits empirically.

Figure 3 illustrates the type of PWM that you have to generate in order to control the position of the RC Servo. Even though only three discrete positions are illustrated; the servos can be controlled continuously between –90 (P_{min}) and 90(P_{max}) degrees.

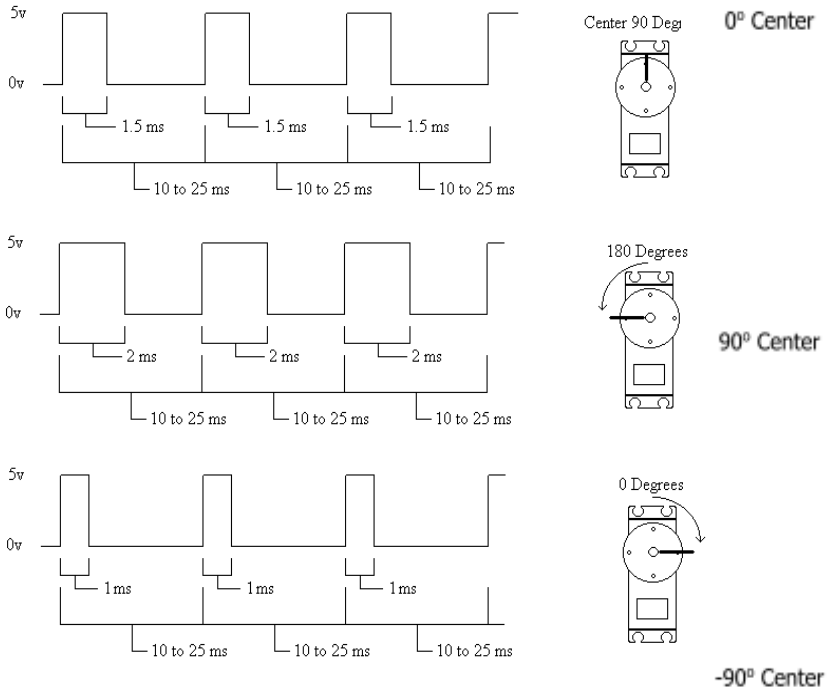

Figure 3. PWM Signal used in controlling a RC servo

STEP 2: Safety first - external regulator

Figure 3 illustrates the theoretical operation of RC servos. However, in practice, the actual values for RC servo limits might deviate from these theoretical ones. Connect the servo pins as follows: middle pin is 5V (mostly supply wire is red), generally the darker colors such as black or brown are used to indicate ground, and the remaining pin is for signaling desired motor position. By the way, use a separate 5V source to feed the servo and make sure that both sources have the same ground. You can get regulated voltage from your PCB supply (again, your turn to figure out how) or you can use a separate regulator such as LM2596 or 7805 for this purpose if you already have. Before you connect the OUTput of the regulator, make sure that you measure the output voltage and guarantee that it is around 5V.
If you have already soldered your PCB and it is operational, you can also use it in this lab.
On the breadboard, the control signal for the motor can be provided from any digital output pin of your PICO. (Even when you use a small size servo or a motor do NOT rely on the Pi Pico regulator, always use an external supply and do not forget to connect grounds!)

STEP 3: Command line control - Long live the REPL

Now, write a program which receives a control position from the REPL command line and converts this into a properly timed signal and **continuously** applies this signal to the RC Servo. The details of this messaging protocol are up to you, but if you send *an invalid value*, Pi Pico should stop sending PWM signals to the motor and hence the motor should be released.

1- Use manual signal generation with digital out and proper sleep sequences.
2- Use a library based solution

STEP 4: Still no GUI: How about a POT

Next step is to control the RC servo using your POT. In other words, your program should map POT output to the RC servo position, and you turn the POT from one end to the other, RC servo should turn in sync with the POT. Your motor is expected to be responsive the the POT changes.

STEP 5: GUI time now

Implement a Python Program on your PC that acts like the screenshot illustrated in Figure 4. You can use one scrollbar or a slider to continuously change servo position and 5 radio buttons to select a preset one of the preset positions. The release button speaks for itself, i.e., after it is pressed, the motor should be freed and you will be able to turn it manually. After motor is released if any other control is used, the RC motor should be active as before.

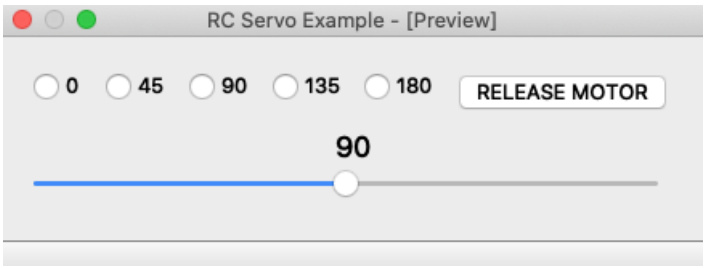

Figure 4. RC Servo controller GUI

RULES of the GAME: First thing last

Have fun!